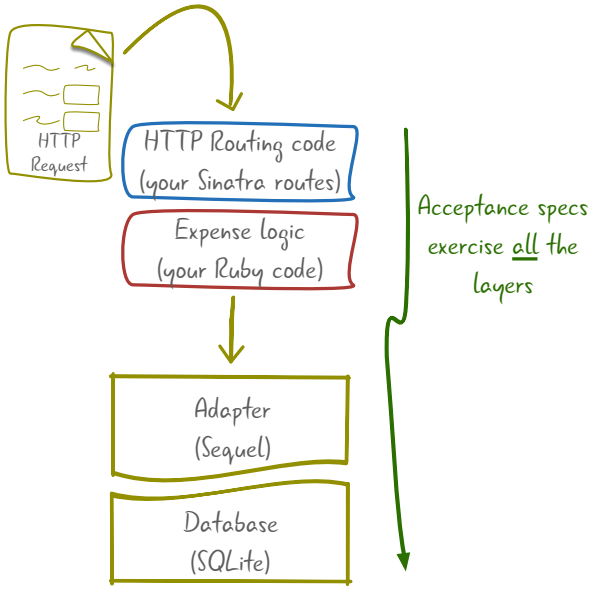
Here are the major parts of the app:

- A web application written in Sinatra that will receive incoming HTTP requests (to add new expenses or search for existing ones)[1]
- A database layer using Sequel to store expenses between requests[2]
- A set of Ruby objects to represent expenses and glue the other pieces together

The following diagram shows how the pieces fit together:



We need to test all of these in different ways. We begin with acceptance specs that drive the entire app from the outermost layer, the HTTP request/response cycle.

Why Not Rails?

We could have used Rails to build this project. However, Rails has a lot of features we don't need here, such as mailers, user-facing views, an asset pipeline, and a job queueing system. Also, Rails sets up a test harness for you. That's useful for real-world projects but gets in the way when you're learning how to configure your own tests.

On the other hand, small JSON APIs like the one we're building here are right in Sinatra's sweet spot. And it's simple enough that you can easily connect it to RSpec on your own. Everything you learn in this book will still apply to Rails projects.

1. http://www.sinatrarb.com/
2. http://sequel.jeremyevans.net/